Retail Store Sales & Inventory Management System

Job Interview Project | SQL Server (SSMS) + Excel Dashboard

A practical analytics solution for a retail store to monitor revenue, profitability, customer activity, and stock replenishment.

Prepared as a runnable demonstration project with synthetic sample data.

1. Executive Summary

This project models a small retail business with 25 products, 15 customers, 245 orders and 587 order lines across 1 January–31 March 2026. The SQL layer provides relational integrity, reusable views, stored procedures, indexing, and an inventory trigger. Excel presents KPI cards, sales trends, category comparisons, and inventory status.

2. Business Problem

Retail teams often track stock and sales across disconnected sheets. This creates duplicated records, inconsistent discount calculations, late reordering, and slow management reporting. The solution centralizes transaction data and standardizes sales and inventory reports.

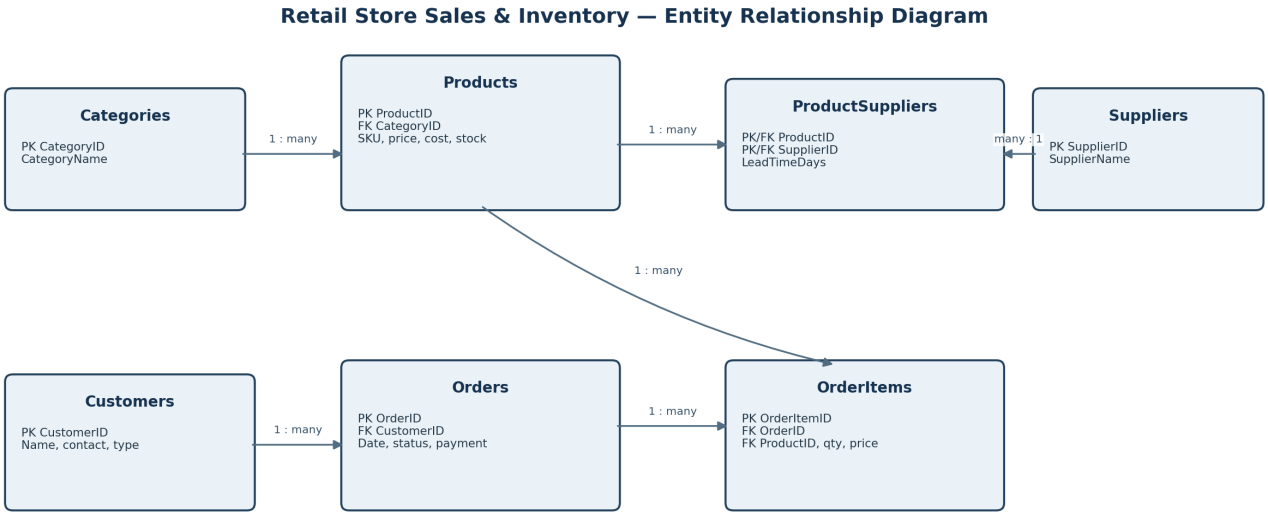
3. Objectives

• Track products, categories, suppliers, customers, orders and line items.
• Calculate net sales, cost and gross profit after discounts.
• Identify best-selling products and high-performing categories.
• Flag stock at or below the reorder level.
• Provide reproducible reports for business decisions.

4. Technology Stack

Microsoft SQL Server 2019+; SQL Server Management Studio; Microsoft Excel or LibreOffice Calc; Python-generated synthetic data and report visuals.

5. Data Model



Core flow: Customers place Orders → OrderItems reference Products. Products are grouped by Categories and sourced through ProductSuppliers.

Categories 1-to-many Products; Products and Suppliers are many-to-many via ProductSuppliers; Customers 1-to-many Orders; Orders 1-to-many OrderItems; Products 1-to-many OrderItems.

6. SQL Implementation

Integrity: primary keys, foreign keys, unique SKU/order number, and check constraints validate quantities, prices, discounts and statuses.
Views: vw_SalesDetail centralizes net sales, cost and gross profit; vw_InventoryStatus labels stock health.
Stored procedures: usp_GetSalesSummary accepts a date range; usp_RecordStockReceipt records a stock increase.
Trigger: completed-order line inserts decrease stock and reject insufficient inventory. The historical seed import disables the trigger to avoid double deduction.

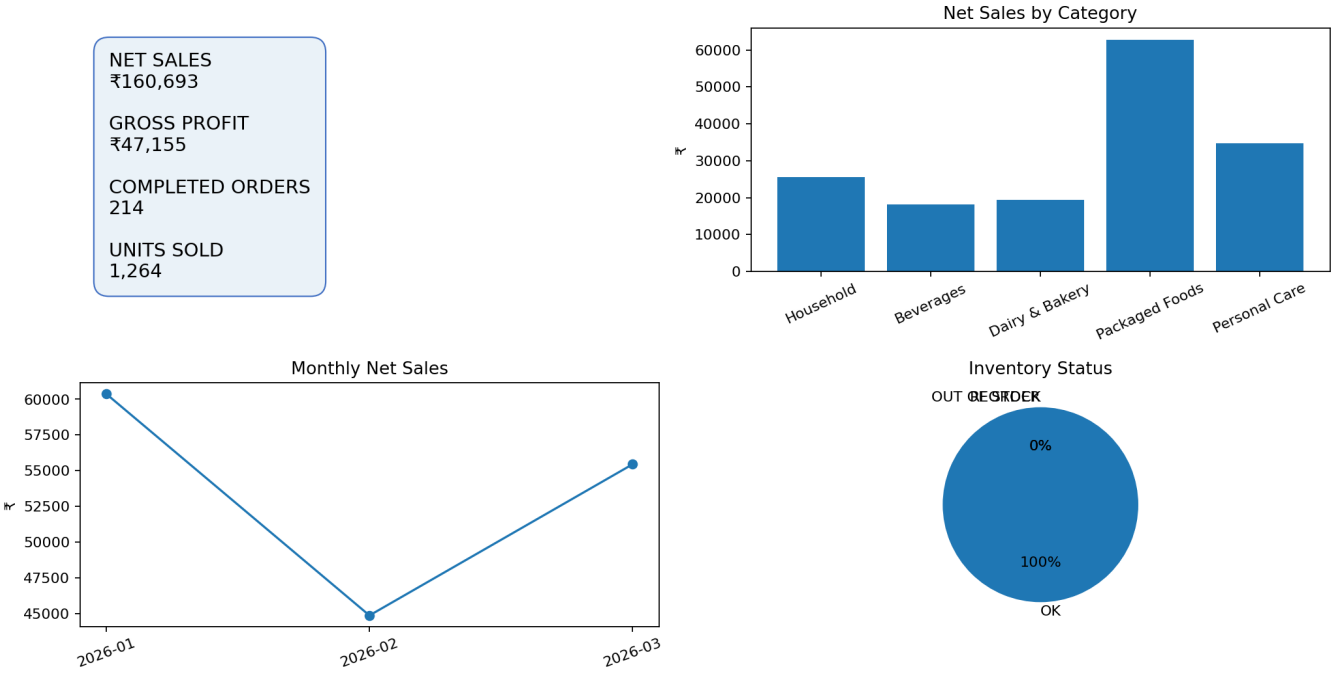
7. Key Business Logic

Net Sales = Quantity × Unit Price × (1 – Line Discount) × (1 – Order Discount).
Cost Amount = Quantity × Unit Cost.
Gross Profit = Net Sales – Cost Amount.
Sales KPIs include Completed orders only. Cancelled and Returned statuses are excluded; refunds are not separately modeled in this demonstration.

8. Example Insights

Using the included sample data, completed-order net sales are approximately ■160,692.58, gross profit is approximately ■47,154.58, 214 distinct completed orders contributed to the item-level sales dataset, and 1264 units were sold. Exact values are also available in the Excel dashboard after formulas recalculate.

Retail Store Dashboard — illustrative sample-data preview



9. How to Run

1. Open SSMS and connect to your local Database Engine.
2. Execute sql/01_Create_Database_and_Schema.sql.
3. Execute sql/02_Load_Sample_Data.sql.
4. Execute sql/03_Analytics_Reports.sql.
5. Open Retail_Store_Dashboard.xlsx in Excel and enable formula recalculation if prompted.

10. Interview Talking Points

• Why separate Orders from OrderItems? One order can contain multiple products, so line-level data belongs in a child table.
• Why use a view? It centralizes business calculations so reports remain consistent.
• Why indexes? Date/status and product filters are common reporting access paths; actual production indexes should be validated against execution plans.
• What would you improve? Inventory movement ledger, transaction-safe order checkout, purchase orders, return/refund tables, row-level permissions, audit logs, data validation, automated tests, and a Power BI dashboard.

11. Limitations and Production Readiness

This is an interview/demo implementation, not a production point-of-sale system. The seed data is synthetic. The trigger covers inserted order items for orders already marked Completed; a real checkout flow must manage status changes, concurrency, cancellation, returns, and stock reservation within transactions. The schema script drops and recreates project tables, so never run it on a database containing valuable data.

12. Deliverables

SQL scripts, CSV data, Excel dashboard, ER diagram, setup guide, and illustrative screenshots are included in the ZIP archive.